INHERITANCE IN C++

CONDUCTED BY

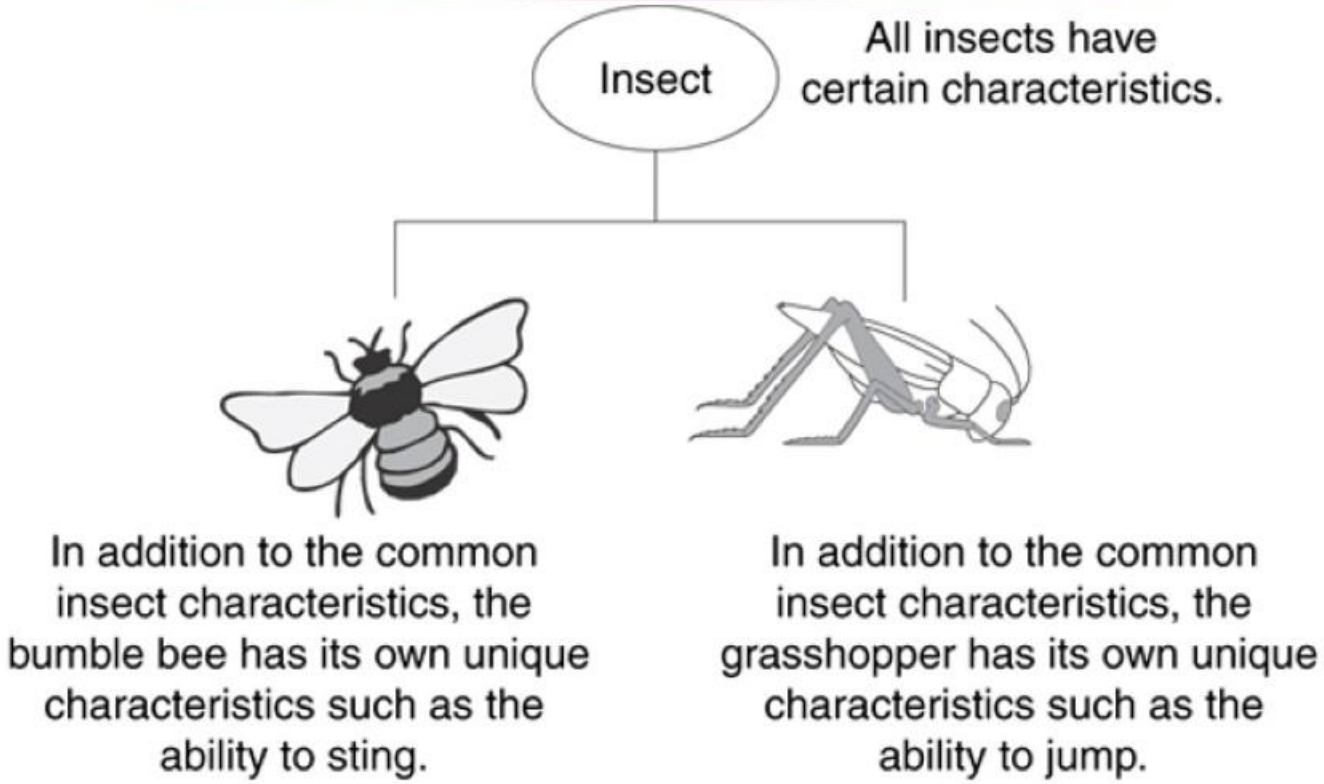
MEHEDI HASAN

LECTURER, DEPARTMENT OF CSE

What is an inheritance?

- Inheritance is the process by which new classes, called derived classes, are created from existing classes, called base classes.
- The derived classes have all the features of the base class and the programmer can choose to add new features specific to the newly created derived class.
- The idea of inheritance implements a relationship. For example, mammal IS-A animal, dog IS-A mammal, hence dog IS-A animal as well, and so on.

Example: Insect Taxonomy



The "is a" Relationship

- Inheritance establishes an "is a" relationship between classes.
  - A cat/dog/Lion is an animal
  - A flower is a plant
  - A football player is an athlete
  - A rose is a flower
  - A parrot is a bird
  - A mango is a fruit
  - A laptop is a computer
  - A train is a transport
  - A guitar is a musical instrument
  - A river is a waterbody
  - A doctor is a professional

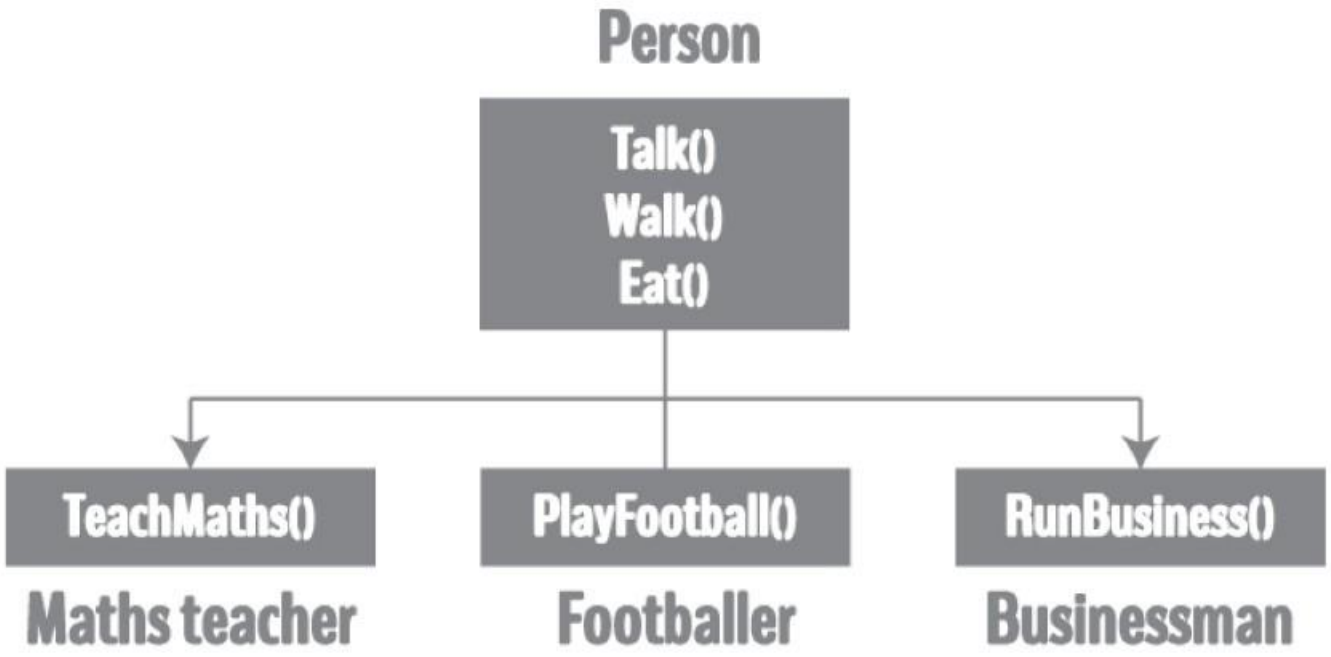
Why should inheritance be used?

- Suppose, in your game, you want three characters - a maths teacher, a footballer and a businessman.
- Since, all of the characters are persons, they can walk and talk. However, they also have some special skills. A maths teacher can **teach maths**, a footballer can **play football** and a businessman can **run a business**.
- You can individually create three classes who can walk, talk and perform their special skill as shown in the figure below.

Without Using Inheritance

Maths teacher	Footballer	Businessman
Talk() Walk() TeachMaths()	Talk() Walk() PlayFootball()	Talk() Walk() RunBusiness()

Using inheritance



FEATURES/ADVANTAGES OF INHERITANCE

- Reusability of code.
- Saves time and effort.
- Faster development, easier maintenance, and easier to extend.
- Capable of expressing the inheritance relationship and its transitive nature, which ensures closeness with real-world problems.

DISADVANTAGES OF INHERITANCE

- Inherited functions work more slowly than normal functions as there is indirection.
- Improper use of inheritance may lead to wrong solutions.
- Often, data members in the base class are left unused, which may lead to memory wastage.
- Inheritance increases the coupling between the base class and derived class.

APPLICATIONS OF INHERITANCE

- The main purpose of introducing inheritance is to "REUSE" your code, and in the same way, when you reuse your code, it means you directly OPTIMIZE your code, and it will reduce the redundancy of your code.
- Inheritance is a fundamental feature of object-oriented programming.

Inheritance – Terminology and Notation in C++

- Base class (or parent) – inherited from
- Derived class (or child) – inherits from the base class
- Notation:

```
class Base
{// base class
  . . .
};
class Derived: public Base
{                    // derived class
  . . .
};
```

```
class Person
{
 ... .. ...
};
class MathsTeacher : public
Person
{
... .. ...
};
class Footballer : public Person
{
.... .. ...
}
```

The types of inheritance in c++

1) Single inheritance.
2) Multiple inheritance.
3) Multilevel inheritance.
4) Hierarchical inheritance.
5) Hybrid inheritance.

Single inheritance

- **Single inheritance:** It is defined as the inheritance in which a derived class is inherited from only one base class. where 'A' is the base class, and 'B' is the derived class.

- An animal is derived from living things
- A car is derived from a vehicle
- Typist is derived from staff

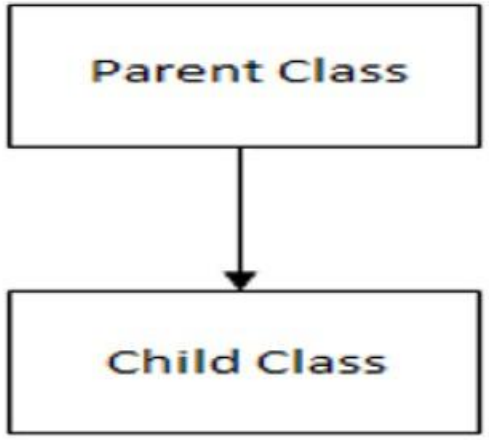


Fig: Single inheritance

Single inheritance: Syntax of Single Inheritance

```
class base_classname {
 properties;
 methods;
};
class derived_classname : visibility_mode base_classname
 {
properties;
methods;
};
```

Single inheritance: Syntax of Single Inheritance

```
class AddData          //Base Class
    {
        protected:
            int num1, num2;
        public:
            void accept()
            {
                cout<<"\n Enter First Number : ";
                cin>>num1;
                cout<<"\n Enter Second Number : ";
                cin>>num2;
            }
    };
    class Addition: public AddData   //Class Addition – Derived Class
    {
            int sum;
        public:
            void add()
            {
                sum = num1 + num2;
            }
            void display()
            {
                cout<<"\n Addition of Two Numbers : "<<sum;
            }
    };
```

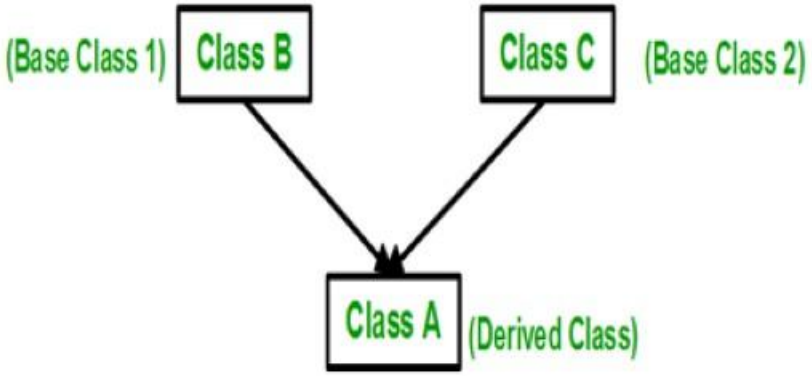
```
int main(){
    Addition a;
    a.accept();
    a.add();
    a.display();
    return 0;
}
```

Multiple inheritance

- **Multiple inheritance:** It is a feature of C++ where a class can inherit from more than one class.

```
class subclass_name : access_mode
base_class1, access_mode base_class2, ....
{
//
body of subclass
};
```

Fig: Syntax of Multiple Inheritance



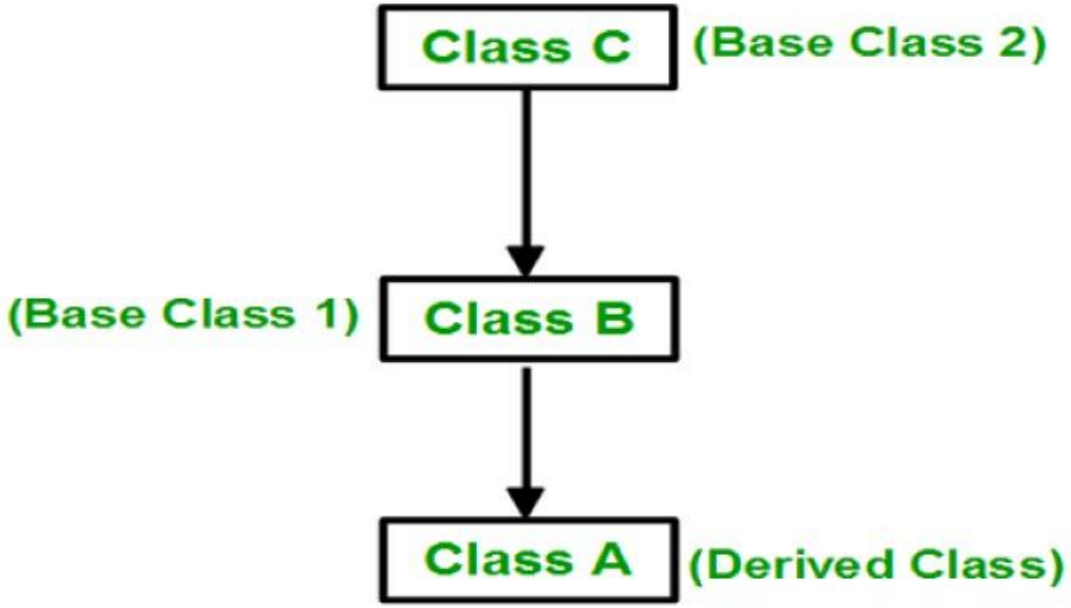
Multiple inheritance

```
// first base class
class Vehicle {
 public:
   Vehicle()
   { cout << "This is a Vehicle" << endl; }
};
 // second base class
class FourWheeler {
 public:
   FourWheeler()
   {
     cout << "This is a 4 wheeler Vehicle" << endl;
   }
};
```

```
// sub class derived from two base classes
class Car: public Vehicle, public FourWheeler {
 };
 // main function
int main()
{
   // creating object of sub class will
   // invoke the constructor of base classes
   Car obj;
   return 0;
}
```

Multilevel inheritance

- **Multilevel inheritance:** If a class is derived from another derived class, then it is called multilevel inheritance. So, in C++ multilevel inheritance, a class has more than one parent class.



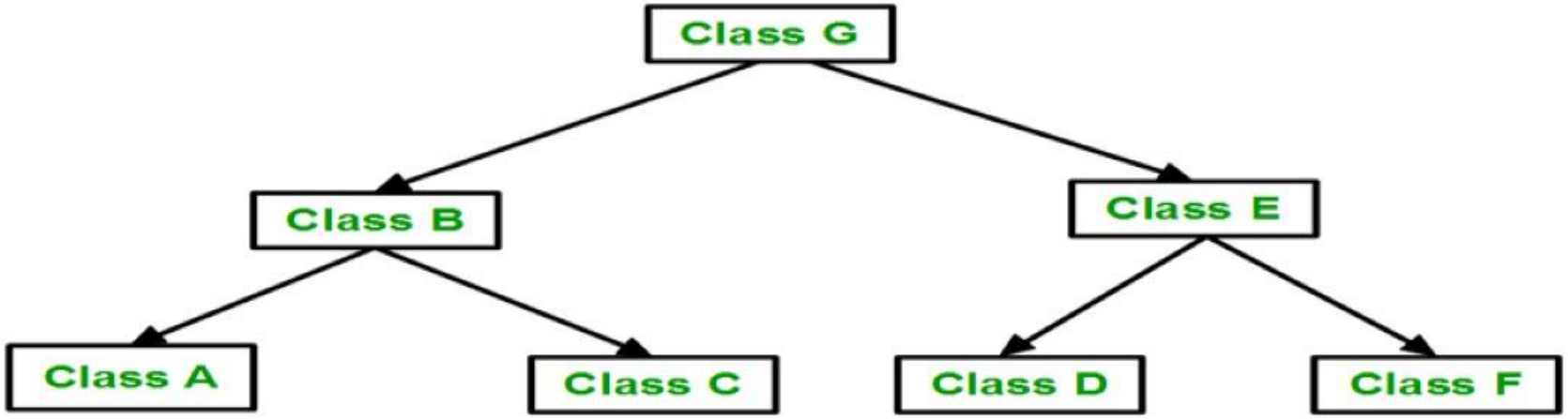
Multilevel inheritance

```
// base class
class Vehicle
{
 public:
   Vehicle()
   {
    cout << "This is a Vehicle" << endl;
   }   };
class fourWheeler: public Vehicle
{  public:
   fourWheeler()
   {
    cout<<"Objects with 4 wheels are vehicles"<<endl;
   }  };
```

```
//subclass derived from two base classes
class Car: public fourWheeler{
  public:
    car()
    {
     cout<<"Car has 4 Wheels"<<endl;
    }  };
// main function
int main()
{
  //Creating an object of a subclass will    invoke the
constructor of the base classes
   Car obj;    return 0;      }
```

Hierarchical inheritance

- **Hierarchical inheritance:** When several classes are derived from a common base class, it is called hierarchical inheritance. In C++ hierarchical inheritance, the feature of the base class is inherited by more than one subclass.



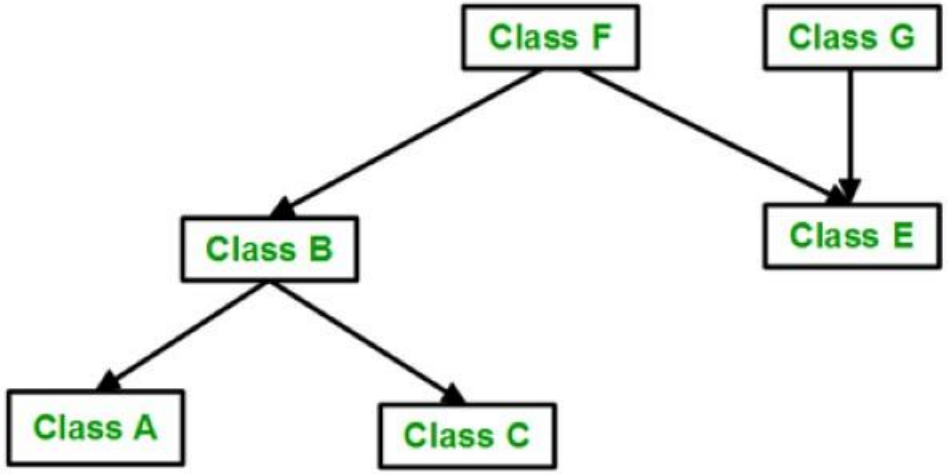
Hierarchical inheritance

```
// base class
class Vehicle
{
 public:
  Vehicle()
  {
   cout << "This is a Vehicle" << endl;
  }   };
 // first sub class
class Car: public Vehicle
{   };
```

```
// second sub class
class Bus: public Vehicle
{   };
 // main function
int main()
{
   // creating object of sub class will
   // invoke the constructor of base class
   Car obj1;
   Bus obj2;
   return 0;
}
```

Hybrid inheritance

- **Hybrid inheritance:** The inheritance in which the derivation of a class involves more than one form of inheritance is called hybrid inheritance. Basically, in C++, hybrid inheritance is a combination of two or more types of inheritance. It can also be called multi-path inheritance.



Hybrid inheritance

```
// base class
class Vehicle
{
 public:
  Vehicle()
  {
   cout << "This is a Vehicle" << endl;
  }
};
```

```
//base class
class Fare
{
   public:
   Fare()
   {
      cout<<"Fare of Vehicle\n";
   }
};
```

Hybrid inheritance

```
// first sub class
class Car: public Vehicle
{
 …
};

// second sub class
class Bus: public Vehicle, public Fare
{
    …
};
```

```
 // main function
int main()
{
   // creating object of sub class will
   // invoke the constructor of base class
   Bus obj2;
   return 0;
}
```